Analyzing VinUni Building Detection Using YOLOv8 Segmentation

COMP4040 – Data Mining and Big Data Analytics

Milestone 1 Report

Team Members:

- Trinh Tuan Hung - V202401654
- Tran Thi Quynh Anh - V202301170

Contributions:

- Trinh Tuan Hung:
  - Designed and implemented the YOLOv8 segmentation training pipeline using Google Colab.
  - Implemented dataset preprocessing scripts, including video frame extraction and prediction visualization.
  - Configured and trained the YOLOv8 segmentation model.
  - Evaluated model performance using precision, recall, and mAP metrics.
  - Developed custom visualization logic for displaying segmentation masks without bounding boxes.
  - Perform quality check annotations.
- Tran Thi Quynh Anh:
  - Collected the Building K dataset using smartphone photography and campus walking videos.
  - Organized and filtered the dataset to remove low-quality and redundant samples.
  - Performed segmentation annotation using Roboflow.
  - Managed dataset preprocessing and augmentation configurations within Roboflow.
  - Contributed to report writing, experimental design, and interpretation of results.

I. Introduction And Motivation

Automatic building recognition is an important computer vision task with applications ranging from augmented reality and navigation systems to service robotics. As VinUniversity continues to grow and expand, there is a need for technologies that can recognize buildings in images and videos to facilitate smart-campus applications such as interactive virtual or real campus tours or autonomous navigation of robotic systems and drones around campus.

At the beginning of the project, this proposal aimed to classify many VinUni buildings using a combination of classical computer vision techniques and machine learning methods. Later, during the project, the scope was adjusted to first build a robust deep learning solution for one building and then scale it up to the whole campus.

Initially we chose to focus on Building K, which is somewhat isolated from the other campus buildings making it easier to acquire clean training images as well as to perform the time consuming segmentation annotation task. Its image appearance is generally distinct from neighbouring buildings allowing us to develop models on images where the building is the primary object without too much visual noise.

We made a few design decisions that significantly simplified the problem. One of them was to use segmentation instead of bounding-box detection. The images we'll be working with in this campus-based project contain a lot of background noise, including trees, vehicles, sidewalks, shadows, and pedestrians. Including all of that extra information in a bounding box would be fruitless. Instead, we can have the model directly draw a segmentation mask of the building. This can be especially helpful for buildings with complex facades, as well as those that are only partially visible in the image.

We employ YOLOv8 segmentation for the detection model, which strikes a good balance between detection quality, segmentation accuracy, real-time speed, and simplicity of the implementation. Unlike heavier segmentation networks, YOLOv8 allows for efficient and fast prototyping and is well-suited for real-time deployments.

II. Dataset Collection

The project utilizes a smartphone camera and records a custom dataset of Building K on VinUniversity campus. Unlike many approaches that rely on publicly available datasets, the project attempts to address real-world scenarios by collecting data on the VinUni's campus.

We started with Building K, which is somewhat independent from the other campus buildings around it, providing a cleaner canvas to both visually annotate and evaluate the model's performance on, as opposed to other buildings heavily intersecting with

neighboring structures. After fine tuning the model on Building K, we observed consistent improvements for subsequent buildings on campus.

The data collection process consisted of:

- manually captured photographs
- walking videos recorded around Building K

Images were intentionally captured from multiple:

- viewpoints
- distances
- camera angles

To make the model more robust we have included a number of scenarios. The scenarios also attempt to include a number of environmental variables such as cloudy, shadowy, highly occluded scenarios in order to improve the models performance in real world scenarios. We have also tried to be more realistic with the campus scenes by including a number of scenarios with a variety of vegetation, scenarios showing straight and uneven sidewalks, scenarios with cars parked on the side walks, etc.

To increase the size of the current dataset, videos were converted into individual images using a python script and OpenCV. These frames were then manually filtered to remove redundant and bad images (e.g. blurry, no visible buildings). The frames were also sampled to avoid overloading the dataset with too many images.

The final dataset contained **512 annotated images.**

The dataset was automatically split using Roboflow into:

- **422 training image** (82%)
- **60 validation images** (12%)
- **30 testing images** (6%)

This separation allows the model to:

- learn from the training set
- tune performance using validation data
- evaluate generalization on unseen test images

III. Data Annotation and Preprocessing Using Roboflow

Our image dataset was annotated via the website Roboflow. At first, we had intended to use bounding-boxes around regions of interest within our images, but after a few test

runs we found that the boxes included way too much noise – i.e. trees, sidewalks, parts of the sky, parked cars and then even additional shadows on the race car itself.

Given time pressure, we moved to a segmentation-based approach to annotation. We created polygon masks to draw outlines of Building K, which better captures the actual building geometry and neglects nuisance objects that a rectangular box would inevitably incorporate. The result is better localization and a cleaner view for a globally distributed team.

Roboflow was additionally used to:

- manage dataset organization -generate dataset splits
- apply preprocessing
- perform automatic augmentation

During preprocessing, all images were:

- automatically orientation-corrected using EXIF metadata
- resized to **640×640 resolution**

We also applied augmentation to improve robustness and to artificially increase the diversity of our dataset. Roboflow returned ~**2 augmented outputs for every original** image.

- 90-degree clockwise rotations
- 90-degree counter-clockwise rotations -upside-down rotations
- grayscale transformations applied to approximately 25% of images

The techniques employed here improve robustness to viewpoint and environmental variation, and prevent overfitting on the small custom dataset.

The dataset has been exported in YOLOv8 segmentation format, i.e., segmentation polygons are presented as normalized coordinates inside corresponding annotation files.

This version doesn't have a model.

Train an optimized, state of the art model with Roboflow or upload a custom trained model to use features like Label Assist and Model Evaluation and deployment options like our auto-scaling API and edge device support.

Train Model

Available Credits: 3

How to Upload Custom Weights

512 Total Images View All Images →



Dataset Split

TRAIN SET 82%	VALID SET 12%	TEST SET 6%
422 Images	60 Images	30 Images

Preprocessing

Auto-Orient: `Applied`

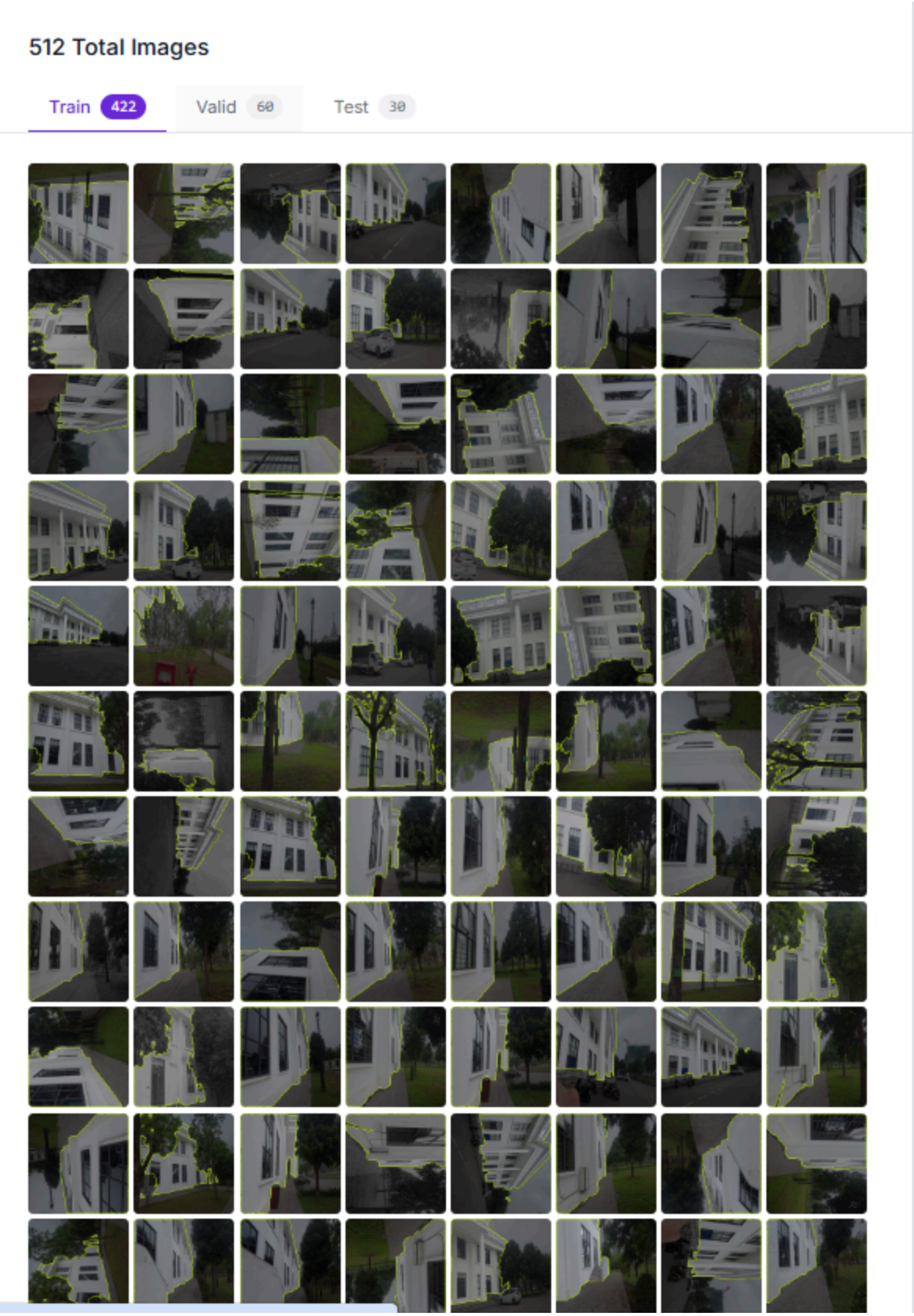
Resize: `Stretch to 640x640`

Augmentations

Outputs per training example: `2`

90° Rotate: `Clockwise, Counter-Clockwise, Upside Down`

Grayscale: `Apply to 25% of images`



IV. Model Selection

After considerable research into different approaches to detect and segment objects, the decision was made to go with YOLOv8 segmentation.

- lightweight architecture
- strong segmentation performance
- fast inference speed
- ease of integration with custom datasets

We started with a pretrained YOLOv8 nano image segmentation model: the model weights were downloaded from the internet and we began training from the file yolov8n-seg.pt. This model uses transfer learning, meaning it starts with a set of learned weights that already know a lot about images of buildings, and then adjusts those weights to better suit the Building K that we're particularly interested in.

Compared to training a model from scratch, the process was much faster and reduced computation requirements.

V. Training Pipeline

For this project, I used Ultralytics YOLOv8 to train the model on Google Colab with a GPU accelerator.

The training process consisted of:

1. Uploading and extracting the dataset
2. Loading the YOLOv8 segmentation model
3. Training for 50 epochs
4. Evaluating segmentation performance on 5. validation data
5. Generating predictions on unseen test images

All images were resized so that they had a resolution of 640×640 pixels prior to model training. The model was then trained utilizing the configuration from data.yaml that was included in the directory with the script.

VI. Segmentation Inference and Visualization

The trained model was evaluated on images that the model had not seen before. Instead of the usual bounding boxes in the output, a custom function was written to display the segmentation masks generated by the model.

The bounding boxes have been removed from the current data release as the boxes were visually coinciding with a large portion of irrelevant background, and a more

accurate way to view the geometry of the buildings is through the segmentation masks provided.

Manual confidence scores have been appended to the prediction outputs to provide detection confidence for users of this API while keeping the output of the API simple to read.

VII. Preliminary Results and Interpretation

The YOLOv8 segmentation model was evaluated on the validation dataset containing 60 images and 179 annotated building instances. The model achieved the following results:

- **Precision:** 0.9967
- **Recal:** 0.5028
- **mAP@50:** 0.5924
- **mAP@50-95:** 0.4893

Precision of around 99.67% was obtained for the model: Building K was annotated in almost all instances where it was present, with only a little false positive annotation and very few instances where trees, sidewalks, adjacent buildings or other elements of the scene were mis-classified as the target building. This supports the use of a segmentation based approach for annotation.

However, the recall score remained at 50.28%, indicating that, while the model was usually correct when it did choose to predict the presence of a building, it did fail to detect many more buildings when conditions were less ideal. Several factors may be limiting the ability of the model to handle these tougher images.

- partial occlusion from trees and vegetation
- extreme side-view camera angles
- parts of the building are cut off from the view
- relatively small dataset size
- uneven viewpoint distribution within the training data

Observing the difference between precision and recall, we note that the current system is somewhat conservative and tends to avoid false alarms at the cost of sometimes failing to correctly classify images where it is confident that the building is prominent. This limitation is also likely influenced not only by dataset diversity, but also by the current dataset split configuration. The dataset currently uses an 82%-12%-6% train-validation-test split, meaning the validation and testing sets remain relatively small. As a result, a small number of difficult images can significantly affect the evaluation metrics.

The score of 0.5924 (mAP@50) suggests a reasonable but not excellent segmentation quality for Building K from different viewpoints. While at 50% the model is able to reliably detect and segment most parts of Building K and achieves high mAP values, the score drops significantly when considering overlap thresholds starting from 55%. Furthermore, the mAP@50-95 score of 0.4893 underlines that the model's accuracy drops with stricter overlap thresholds, i.e. the mask boundaries cannot exactly follow the true edges of the building.

Even with the aforementioned issues, visual inspection of results shows the network is able to produce reasonable segmentation masks for Building K under different viewpoints and lighting conditions. In a majority of the tested images, the network is able to segment Building K while ignoring large amounts of irrelevant background such as vegetation, cars, and sidewalks. Therefore, the approach of image segmentation instead of object detection using bounding boxes is sufficient for this task.

In addition, our results demonstrate an important role of dataset quality and consistency in segmentation accuracy. Future work will focus on

- increasing dataset size
- collecting more diverse viewpoints
- improving annotation consistency
- balancing lighting and occlusion conditions
- training with larger YOLOv8 segmentation variants

Overall, We have achieved an important milestone where we can now run end-to-end segmentation, from images to masks, through a pipeline.

- custom dataset collection
- segmentation annotation
- YOLOv8 segmentation training
- qualitative visualization
- quantitative evaluation

Despite the somewhat poor performance, the current setup already provides a very good base for future expansion (to more buildings of VinUni, e.g., to campus-wide recognition task).

VIII. Future Work

The next phase of the project will focus on:

- expanding the dataset size

- improving annotation consistency
- testing additional VinUni buildings
- comparing segmentation performance across buildings
- experimenting with larger YOLOv8 architectures

We also plan to revisit classical computer vision techniques (given current dataset) and compare them to deep-learning approaches.

Additionally, future experiments may explore alternative dataset split ratios such as 70-